With our gem cache populated, let's try installing a new gem. Imagine we want to add Devise for authentication. Let's add it to our Gemfile:

```
...
gem 'redis', '~> 4.0'
# Authentication
gem 'devise', '~> 4.4', '>= 4.4.1'
...
```

OK. Now, like we'd typically do in a non-Dockerized environment, we install the gem by running the bundle install command:

```
$ docker-compose exec web bundle install
```

You should see output similar to the following:

```
The dependency tzinfo-data (>= 0) will be unused by any of the platforms
Bundler is installing for. Bundler is installing for ruby but the dependency
is only for x86-mingw32, x86-mswin32, x64-mingw32, java. To add those plat-
forms to the bundle, run `bundle lock --add-platform x86-mingw32 x86-mswin32
x64-mingw32 java`.
Fetching gem metadata from https://rubygems.org/.........
Fetching gem metadata from https://rubygems.org/.
Resolving dependencies.....
Using rake 12.3.2
Using concurrent-ruby 1.1.4
«...»
Fetching warden 1.2.8
Installing warden 1.2.8
Fetching devise 4.5.0
Installing devise 4.5.0
«...»
Using turbolinks-source 5.2.0
Using turbolinks 5.2.0
Using uglifier 4.1.20
Using web-console 3.7.0
Using webpacker 3.5.5
Bundle complete! 21 Gemfile dependencies, 90 gems now installed.
Bundled gems are installed into `/gems`
```

The output shows that Devise is the only gem that's installed—the others are reused from our gem cache.

There's one final complication: our webpack_dev_server service. Since this uses the same Dockerfile as our web service, we also need to rebuild its image:

```
$ docker-compose build webpack_dev_server
```

Similarly, we then need to recreate the webpack_dev_server container from the new image: